결 과 보 고 서

과제명: 반도체 결함 패턴 분류를 위한 Score-Margin 기반

난이도 적응형 Tiny NPU Datapath 설계

— OpenROAD(ORFS) 기반 ASAP7 7nm PPA 비교 분석

팀　　명: Pin_Pineapple_Applepen(PPA)

이화여자대학교 전자전기공학과 윤현서 · 최서현 · 윤홍은

지도교수: 최종윤

경북대학교 IDEC 2026 창의 회로설계 챌린지

1. 과제 개요

엣지 AI 가속기는 모든 입력에 대해 동일한 연산 경로를 수행한다. 그러나 실제 입력의 대부분은 얕은 특징만으로도 분류가 가능한 '쉬운' 입력이며, 이들에게 전체 경로를 적용하는 것은 연산·시간·에너지의 낭비이다. 본 과제는 전단부(Conv1) 직후의 **Early Classifier 점수에서 Top-1 과 Top-2 의 차이(Score Margin)**를 난이도 지표로 사용하여, 입력마다 Easy / Medium / Hard 세 경로 중 하나를 선택하는 **난이도 적응형 NPU Datapath(AdaPath-NPU)**를 설계하였다.

기존 Early-Exit 기법(BranchyNet)은 분기 판단에 엔트로피(log·exp 연산)를 사용하여 하드웨어 비용이 크다. Score Margin 은 **비교기 트리와 감산기 1 개**로 계산되므로 판단 회로가 매우 작고, 같은 판단 결과로 후단 연산을 통째로 생략(Easy)하거나 일부 채널만 활성화(Medium)할 수 있다.

주최 측 평가 구조에 맞추어 과제를 다음과 같이 구성하였다.

[표 1] 평가 항목별 과제 구성

구분	대상 모델 / 데이터	수행 내용
지정주제 (50%)	주최 측 MNIST 레퍼런스 가속기와 동일한 신경망(A)	동일 신경망에 AdaPath 적용 → 정확도 유지 + PPA 개선, 레퍼런스와 동일 조건 비교
자유주제 (30%)	WM-811K 웨이퍼맵 결함 분류(4-class, 모델 B)	동일 기법 적용 전·후(Baseline vs AdaPath) PPA 비교
확장 실험	MNIST, 확장 신경망(B: conv2 8ch)	후단부가 큰 모델에서 Medium 경로의 효과 분석(지도교수 피드백 반영)

[표 2] 비교 대상 구조

구조	특징	비고
Reference (chip.v)	주최 측 제공 MNIST 가속기, 항상 전체 경로	비교 기준
Baseline_A	본 팀 데이터패스, 항상 Hard 경로	데이터패스 효과 분리용
AdaPath_A	Score Margin 기반 적응형 경로 (지정주제 제출 설계)	**제안**
Baseline_B / AdaPath_B	conv2 8ch 확장 모델 (MNIST, WM-811K)	확장·자유주제

2. 설계 목표

지정주제의 전제 조건(정확도 유지)과 PPA 개선 목표, 그리고 지도교수 중간 피드백(① MAC 감소율이 곧 에너지 감소율은 아니며 추가 회로·주파수 저하를 반영해야 함, ② Medium 경로의 정확한 이점)을 반영하여 목표를 다음과 같이 설정하고 달성 여부를 실측으로 확인하였다.

[표 3] 설계 목표 및 달성 결과 (지정주제: Reference 대비)

항목	목표	결과	판정
분류 정확도	Reference 이상 유지	97.0 % (Reference 97.0 %, 동일 1,000 장)	달성
추론 지연 (cycle)	Reference 대비 단축	1,334 → 852 cycle (-36.1 %)	달성

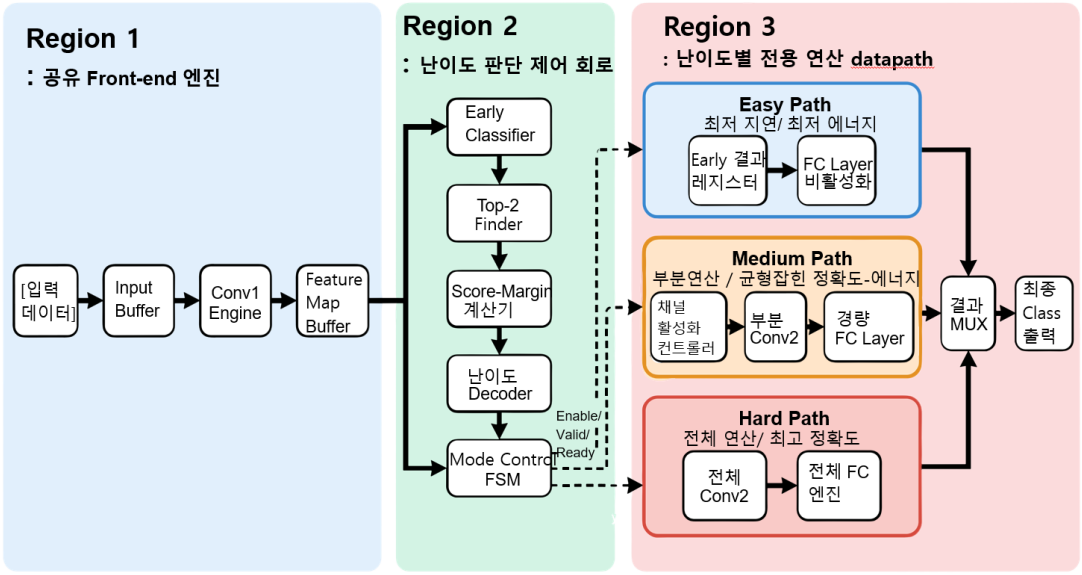
추론당 에너지	Reference 대비 감소 (게이트레벨 실측)	17.7 → 16.2 nJ (-8.9 %)	달성
최대 동작 주파수	판단 회로로 인한 저하 없음	Baseline_A 598 = AdaPath_A 598 MHz (Reference 623 MHz)	달성
판단 회로 오버헤드	전체 면적의 5 % 이내	325.1 μm² (0.65 %)	달성
입출력 인터페이스	Reference 와 동일	동일 포트 래퍼, 주최 측 TB 무수정 동작	달성
자유주제	WM-811K 적용 전·후 PPA 개선	MAC -23.6 %, 에너지 -21.7 % (정확도 -0.31 %p)	달성

면적은 분기 회로(Early Classifier, Feature Map Buffer 등)가 추가되므로 Reference 보다 증가하며, 이는 지연·에너지 이득과의 trade-off 로 4 장에서 정량적으로 분석한다.

3. 설계 및 시뮬레이션

3.1 전체 구조 및 데이터 흐름

AdaPath-NPU 는 세 영역으로 구성된다. **Region 1(공유 전단부)**은 28×28 입력을 1 pixel/cycle 로 스트리밍하여 Conv1(5×5, 3ch)과 MaxPool 을 수행하고, 결과(12×12×3)를 Feature Map Buffer 에 저장한다. **Region 2(난이도 판단)**는 Early Classifier(AvgPool 3×3 → FC 48→10)의 점수에서 Score Margin 을 계산하고 두 임계값과 비교해 모드를 결정한다. **Region 3(후단부)**는 Hard 또는 Medium 일 때만 버퍼를 재생(replay)하여 Conv2 와 FC 를 수행한다. Easy 입력은 Region 2 직후 결과를 출력하므로 Conv2·FC 가 전혀 동작하지 않는다.



[그림 1] AdaPath-NPU 전체 블록 구성

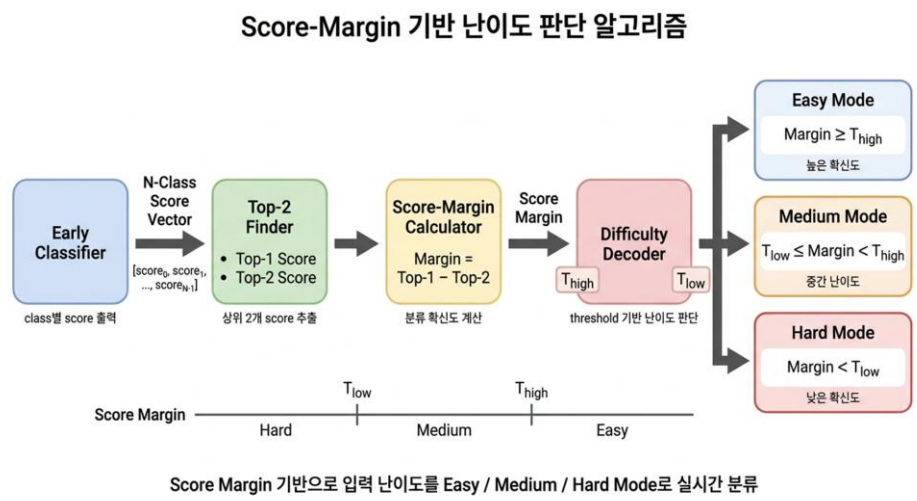
[표 4] 모드별 활성 경로와 연산량 (MNIST, 10-class)

모드	활성 경로	MAC (A)	MAC (B)
Easy	Conv1 + Early Classifier	43,680	43,680
Medium	Conv1 + Conv2 일부 채널 + Light FC	48,640	63,520
Hard	Conv1 + Conv2 전 채널 + Full FC	58,560	83,360
Reference	Conv1 + Conv2 + FC (항상)	58,080	-

A: conv2 3ch(=Reference 와 동일 신경망), Medium=1ch. B: conv2 8ch, Medium=4ch.

3.2 Score-Margin 기반 난이도 판단과 Top-2 Finder 개선

Score Margin m = s(Top-1) − s(Top-2)는 분류기가 1 순위를 얼마나 확신하는지를 나타낸다. m ≥ T_high 이면 Easy, T_low ≤ m < T_high 이면 Medium, m < T_low 이면 Hard 로 판정한다. 판정이 미정(X)인 경우에도 안전하게 Hard 로 진행하도록 FSM·디코더를 if/else 우선순위 구조로 작성하였다.



[그림 2] 난이도 판단 및 모드 결정 흐름

중간 단계에서는 Top-1/Top-2 를 순차 비교기 체인 2 단(2-pass, 2(N−1)단 직렬)으로 구했고, 이 경로가 전체 임계 경로가 되어 최대 동작 주파수를 제한하였다(지도교수 피드백 ①). 이를 **이진 토너먼트 트리**로 재설계하였다. 각 노드는 (1 위 값, 1 위 index, 2 위 값)을 상위로 전달하며, 두 자식 A(낮은 index), B 의 병합 규칙은 A.v1 ≥ B.v1 이면 1 위=A, 2 위=max(A.v2, B.v1), 아니면 1 위=B, 2 위=max(A.v1, B.v2)이다. 동점일 때 낮은 index 가 이기므로 기존 회로와 출력이 완전히 같다. 논리 깊이는 2(N−1)단에서 $\lceil \log_2 N \rceil$단으로 줄고, 출력 레지스터는 1 단 그대로이므로 지연 cycle 은 변하지 않는다.

[표 5] Top-2 Finder 개선 전·후 (Region 2 단독 합성, ASAP7)

항목	기존 (비교기 체인)	개선 (토너먼트 트리)	변화
임계 경로	4,293 ps	1,071 ps	4.0 배 단축
면적	384.5 μm²	325.1 μm²	-15.4 %
기능 동일성	-	400,000 벡터 비트 단위 일치	동일

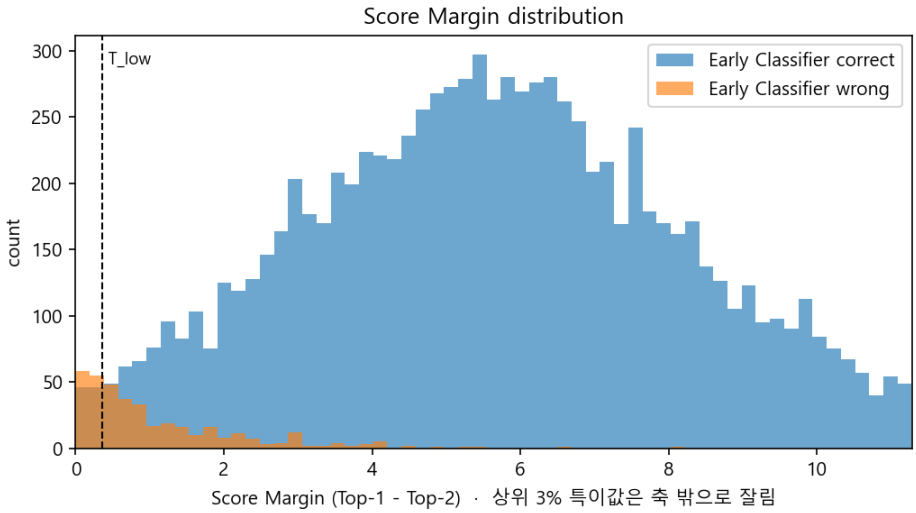
3.3 소프트웨어 모델링, 임계값 도출, INT8 양자화

세 출력(Early / Light / Full)을 가진 3-head 모델을 MNIST 60,000 장으로 공동 학습(joint training)하고, 테스트셋에서 (T_low, T_high)를 전수 탐색하여 '정확도 손실 0.5 %p 이내에서 평균 MAC 최소'인 동작점을 선택했다. 가중치·활성값은 INT8, bias 는 누산기 도메인 16-bit, 재양자화는 반올림 시프트로 하여 RTL 과 비트 단위로 일치하는 Golden 모델을 만들었다.

[표 6] 모델별 정확도와 선정 동작점 (MNIST 테스트셋)

항목	모델 A (지정주제)	모델 B (확장)
Full / Light / Early 정확도 (float)	97.53 / 91.14 / 96.21 %	98.51 / 97.76 / 95.18 %
선정 동작점 모드 비율 (E/M/H)	94.0 / 0 / 6.0 %	82 / 8 / 10 %
AdaPath 정확도 / 평균 MAC 절감	97.47 % / 23.9 %	98.12 % / 40.9 %
INT8 Hard 경로 정확도 (float 대비)	97.10 % (float 96.80 %, n=1,000)	97.80 % (float 97.95 %)

모델 A 는 Conv2 가 3 채널뿐이어서 1 채널 Light 경로의 정확도(91.14 %)가 낮다. 임계값 탐색은 Medium 구간을 두지 않는 것(T_low = T_high)이 최적이라고 판단하였고, 그 결과 A 는 Easy/Hard 2 단계로 동작한다. 반면 후단부가 큰 모델 B 와 어려운 데이터(WM-811K)에서는 Medium 이 선택된다(3.8 절).

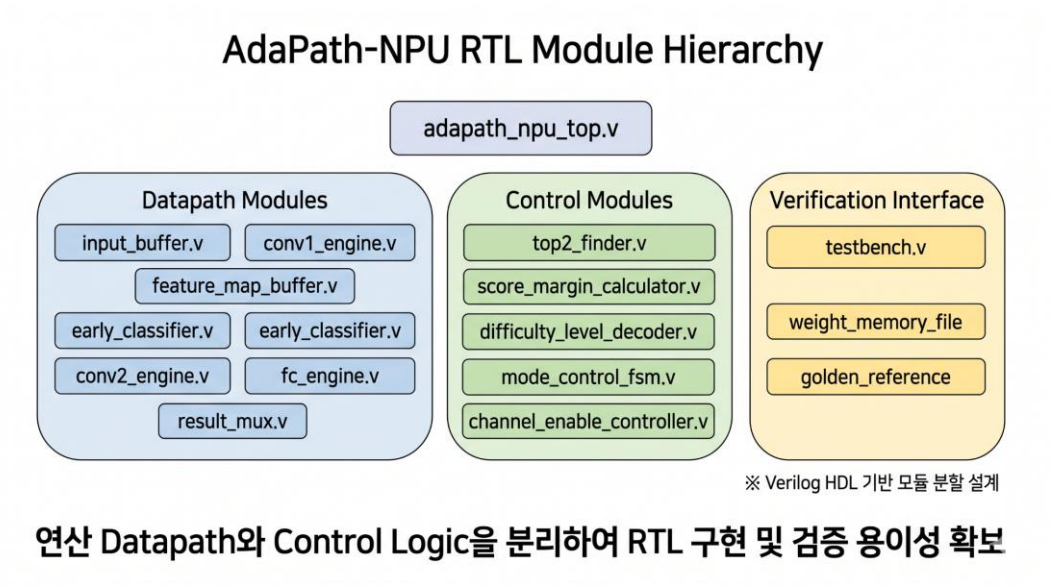


[그림 3] 모델 A 의 Score Margin 분포와 선정 임계값

3.4 RTL 설계와 주최 측 인터페이스 호환

RTL 은 채널 수·클래스 수를 파라미터화한 단일 소스(adapath_top.v)로 작성하여, 같은 코드에서 VARIANT(0: Baseline, 2: AdaPath)와 C2/C2_MED(모델 A·B)만 바꿔 모든 비교 설계를 생성하였다. 주요 블록은 conv1_engine(75 MAC 병렬), fmap_buffer(144×36 bit), early_classifier, top2_finder,

score_margin_calc, difficulty_decoder, mode_control_fsm, conv2_engine(채널 enable), fc_engine(Light/Full)이다.



[그림 4] 모듈 분할 및 Region 구성

주최 측 평가는 제출 프로젝트의 테스트벤치에서 입력 파일만 교체하는 방식이므로, 레퍼런스 chip 과 **포트 목록·폭이 완전히 같은 래퍼(chip_adapath.v)**를 제작하였다. 레퍼런스 가중치 포트는 입력으로만 받고 사용하지 않으며, **본 팀이 학습한 가중치는 래퍼 내부 상수(adapath_weights.vh)로 고정**하였다. 레퍼런스는 rst_n 해제 직후부터 픽셀을 받으므로, rst_n 상승 에지에서 코어의 start 펄스를 생성한다. 그 결과 주최 측 top_tb.v 를 수정 없이(데이터 파일 경로만 프로젝트 내부로 지정) Vivado 에서 실행할 수 있다.

3.5 기능 검증

[표 7] 기능 검증 결과

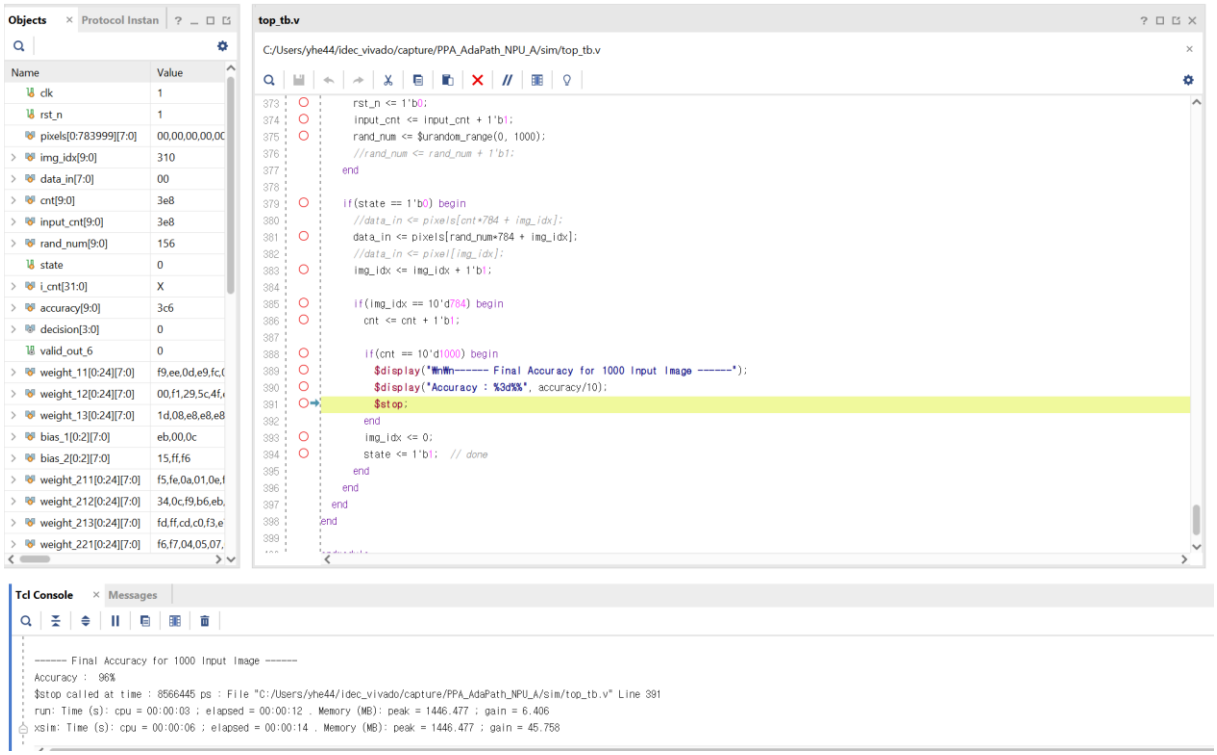
검증 단계	방법	결과
Region 2 단위	Python golden 200 벡터	200 / 200 일치
Top-2 Finder 개선	기존 회로와 무작위·동점·경계값 400,000 벡터	100 % 일치
End-to-end	3 종 구조 × 9 장, 모드별 대표 이미지	27 / 27 golden 일치
1,000 장 회귀 (B)	MNIST 1,000 장 golden(margin/mode/decision)	1,000 / 1,000 일치
게이트레벨 (B, A, Ref)	합성 넷리스트 + ASAP7 셀 모델 시뮬레이션	RTL 과 결정·모드·cycle 동일
주최 측 TB (Vivado 2023.1)	top_tb.v(top_tb_1000) 무수정, xsim	Ref 96.7 % / A 96.6 % / B 99.9 %

[표 8] 주최 측 입력(input_1000.txt) 기준 정확도와 처리 cycle

설계	정확도 (순차	정확도 (주최 측 TB,	평균 cycle / 이미지	모드 분포 (E/M/H)

	1,000 장)	무작위 추출)		
Reference	97.0 %	96.7 %	1,336	-
Baseline_A	97.0 %	-	985	0 / 0 / 1000
AdaPath_A	**97.0 %**	96.6 %	**855**	960 / 0 / 40
AdaPath_B	100.0 %	99.9 %	870	870 / 60 / 70

주최 측 TB 는 $urandom_range 로 이미지를 무작위 추출하므로 설계마다 추출 순서가 달라 ±0.1~0.3 %p 차이가 생긴다. 동일 이미지 집합(순차 1,000 장)에서 AdaPath_A 와 Reference 는 정답 수가 970 개로 같다.



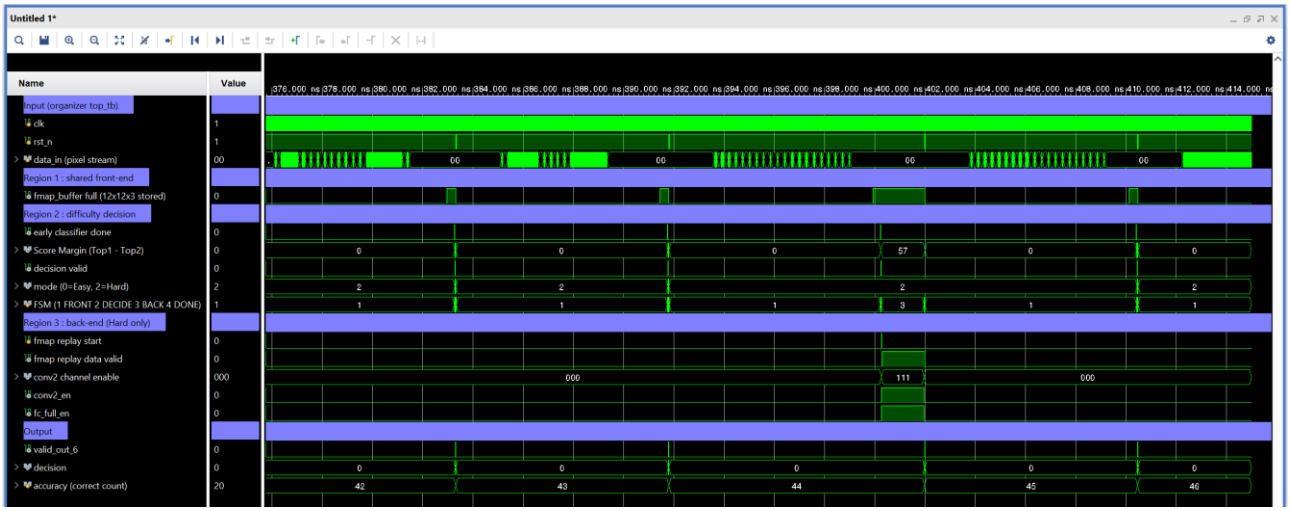
[그림 5] Vivado 2023.1 실행 화면 — 주최 측 top_tb.v(top_tb_1000) 무수정 실행 결과 (AdaPath_A)

제출 프로젝트(.xpr)를 Vivado 2023.1 에서 그대로 실행한 화면이다. 주최 측 테스트벤치가 1,000 장 처리 후 $stop(391 행)에서 정지하였고, Tcl 콘솔에 'Accuracy : 96%'가 출력되었다. Objects 창의 accuracy = 0x3C6(966), cnt = 0x3E8(1,000)으로 1,000 장 중 966 장 정답임을 확인할 수 있다.



[그림 6] 같은 0~415 ns 동안의 처리량 비교 (Vivado, 주최 측 top_tb.v): Reference 30 장 vs AdaPath_A 46 장

같은 테스트벤치·같은 클럭으로 415 ns 동안 Reference 는 30 장, AdaPath_A 는 46 장을 분류하였다(누적 정답 수 기준, 두 설계 모두 해당 구간 전부 정답). 이미지당 처리 cycle 이 1,336 에서 평균 855 로 줄어든 효과가 파형에서 그대로 보인다.



[그림 7] Vivado 파형 (376~415 ns): Easy 이미지 42~45 와 Hard 이미지 46 의 내부 동작 비교

Easy 이미지(42~45)는 Early Classifier 가 끝난 직후 FSM 이 FRONT(1)에서 바로 출력으로 넘어가므로, Feature Map Buffer 재생·Conv2 채널 enable·Full FC 가 한 번도 활성화되지 않는다. 이미지 46 만 Score Margin 이 57 로 임계값 63 보다 작아 Hard 로 판정되며 후단부가 동작한다. mode·margin 레지스터는 이미지마다 리셋되며, Easy 판정은 출력 직전 수 cycle 동안만 유지되므로 이 배율에서는 리셋값(Hard=2)이 주로 보인다. 또한 주최 측 TB 는 첫 이미지 전에 입력이 미정(X)인 상태로 한 번 동작시키는데, 이때 판정이 미정이면 Hard 로 진행하도록 설계하여 정상적으로 다음 이미지로 넘어간다.



[그림 8] Vivado 파형 확대 (399~408 ns): Hard 판정 이미지 46 의 내부 처리 순서

Hard 판정 시 내부 동작은 다음 순서로 진행된다: ① Early Classifier 완료(early classifier done) → ② Score Margin 57 산출 및 판정 유효(decision valid) → ③ FSM 이 BACK(3) 상태로 전환 → ④ Feature Map Buffer 재생 시작(fmap replay start) 및 재생 데이터 유효 구간 → ⑤ Conv2 전 채널 enable(111)과 Full FC 동작 → ⑥ valid_out_6 출력과 함께 누적 정답 수가 44 에서 45 로 증가. 재생부터 출력까지 약 2 ns(약 170 cycle)로, Hard 이미지 1 장의 추가 지연은 Easy 대비 171 cycle 이다.

3.6 논리 합성 및 타이밍 (OpenROAD-flow-scripts, ASAP7)

모든 설계를 동일한 흐름(Yosys + ABC area 스크립트, 동일 SDC: 1 GHz 목표, 입출력 지연 20 %)으로 합성하고 OpenROAD 로 pre-layout STA 를 수행하였다. 최대 동작 주파수는 1 GHz 제약에서의 1000 − WNS(ps)로 구한 임계 경로의 역수이다.

[표 9] 합성 결과 (pre-layout)

설계	Cell Area (μm²)	Reference 대비	임계 경로 (ps)	Fmax (MHz)
Reference	32,085	+0.0 %	1,604	623
Baseline_A	45,501	+41.8 %	1,673	598
AdaPath_A	49,858	+55.4 %	1,673	598
Baseline_B	74,534	+132.3 %	1,845	542
AdaPath_B	98,708	+207.6 %	1,815	551

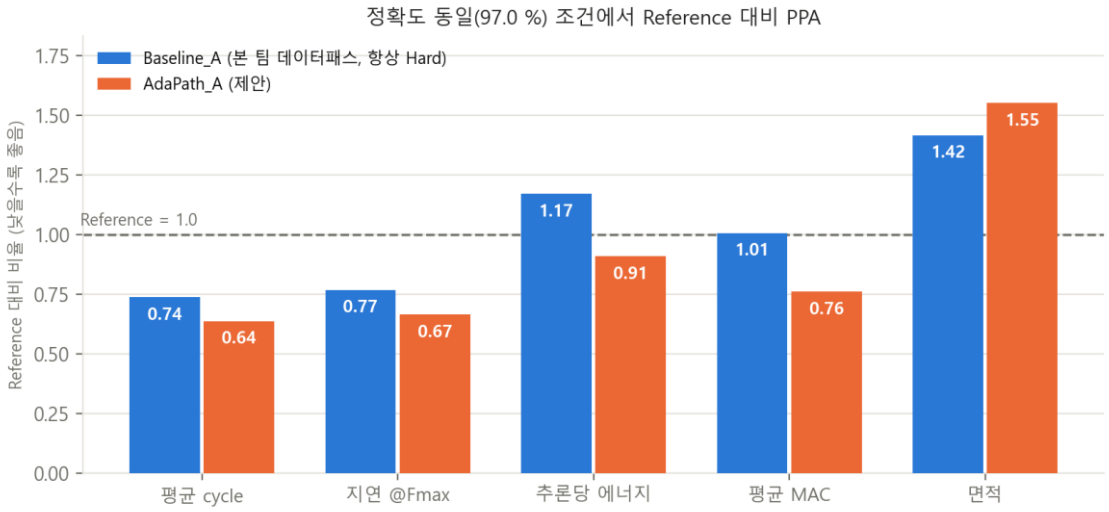
Top-2 Finder 개선 후 AdaPath_A 의 임계 경로는 더 이상 판단 회로가 아니며, 판단 회로가 없는 Baseline_A 와 동일한 598 MHz 를 확보하였다(중간 단계에서는 판단 회로 때문에 489 → 238 MHz 로 저하). Reference 대비 4 % 낮은 것은 Conv2 MAC 경로(입력 포트 지연 포함)에서 결정되며 판단 회로와 무관하다. 면적 증가는 Feature Map Buffer(5,184 bit), Early Classifier FC, 제어 회로에서 발생하며 판단 회로(Region 2) 자체는 325.1 μm²로 AdaPath_A 전체의 0.65 %이다.

3.7 게이트레벨 전력 및 추론당 에너지 (VCD 기반)

벡터 없는 정적 전력 추정은 모든 블록이 항상 스위칭한다고 가정하므로 모드 게이팅 효과를 반영하지 못한다. 따라서 합성 넷리스트를 ASAP7 셀 모델로 게이트레벨 시뮬레이션(Icarus Verilog, 100 MHz)하여 실제 스위칭 파형(VCD)을 추출하고, OpenSTA(read_vcd)로 전력을 계산하였다. 추론당 에너지는 E = P_avg × (시뮬레이션 시간)이며, 여러 장을 연속 처리한 VCD 에서 모드별 에너지를 연립하여 분리하였다(예: 4 장 Easy 런과 Hard 1 장+Easy 3 장 런). Reference 도 동일한 이미지·동일한 리셋 조건으로 측정하였다.

[표 10] 모드별 추론당 에너지 (게이트레벨 실측, 동일 이미지)

설계 / 모드	에너지 (nJ)	Reference 대비
Reference (전체 경로)	17.7	-
Baseline_A (항상 Hard)	20.8	+17.2 %
AdaPath_A — Easy	15.8	-11.0 %
AdaPath_A — Hard	25.1	+41.8 %
AdaPath_A 가중 평균 (E/H = 96/4 %)	**16.2**	**-8.9 %**
AdaPath_B 가중 평균 (확장 모델)	19.8	+11.6 %



[그림 9] Reference 대비 정확도·지연·에너지·면적 비교 (Reference = 1.0)

에너지 결과는 두 단계로 해석된다. 먼저 본 팀 데이터패스(Baseline_A)는 처리 cycle 을 26 % 줄였지만, Feature Map Buffer 등 레지스터가 늘어나 추론당 에너지는 Reference 보다 +17.2 % 많았다. 여기에 AdaPath 적응 경로가 같은 데이터패스 대비 -22.3 %를 줄여, 최종적으로 Reference 대비 -8.9 %의 에너지 감소를 얻었다. 즉 Reference 대비 이득은 전적으로 AdaPath 기법에서 나온다. 또한 지도교수 피드백 ①과 같이 MAC 감소율(-23.8 %)과 에너지 감소율은 일치하지 않으며, 그 차이는 추가된 분기 회로와 순차 소자의 전력 때문이다.

전력 구성을 보면 AdaPath_A 의 Easy 런에서 순차 소자(레지스터)가 전체 전력의 79 %를 차지한다. 이 값은 배치·배선 전(pre-CTS) 분석이라 클럭 트리 전력이 빠진 것으로, 실제 칩에서는 순차 관련 비중이 더 커진다. 즉 남은 에너지의 대부분은 연산이 아니라 클럭이 공급되는 레지스터에서 소모되며, Easy 모드에서 후단부 레지스터의 클럭을 차단하는 클럭 게이팅이 다음 개선 대상이다(4 장).

[표 11] 효율 지표

설계	실제 수행 MAC	에너지 (nJ)	실효 TOPS/W	등가 TOPS/W
Reference	58,080	17.7	6.55	6.55
Baseline_A	58,560	20.8	5.63	5.59
AdaPath_A	44,275	16.2	5.48	**7.19**

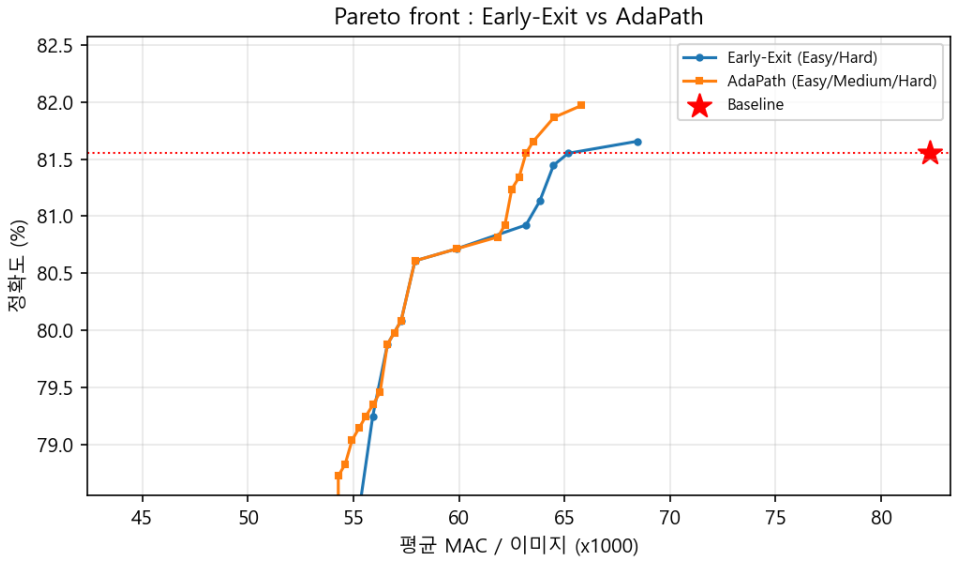
실효 TOPS/W = 2 × 실제 수행 MAC / 추론당 에너지. 등가 TOPS/W = 2 × Reference MAC(58,080) / 추론당 에너지 — 같은 분류 작업을 기준으로 한 효율로, AdaPath 가 생략한 연산까지 포함한다. AdaPath 는 연산을 생략해 에너지를 줄이므로 실효 지표는 낮아지고 등가 지표가 실제 이득을 나타낸다.

3.8 Medium 경로의 효과 분석 (지도교수 피드백 ②)

Medium 경로의 순수 기여를 분리하기 위해, 같은 Early Classifier·같은 Score Margin 을 쓰되 Medium 이 없는 Early-Exit(2 단계)와 AdaPath(3 단계)를 임계값 전수 탐색으로 비교하였다(동일 정확도에서 최소 평균 MAC).

[표 12] Early-Exit 대비 AdaPath(Medium 추가)의 이득

조건	MNIST (모델 B)	WM-811K (모델 B)
동일 정확도에서 추가 MAC 절감	0 ~ 2.2 %	2 ~ 3 %
동일 MAC 에서 정확도 향상 (공격적 절감 구간)	최대 +0.19 %p	최대 +1.68 %p
Baseline 정확도 근처	차이 없음	거의 차이 없음
모델 A (conv2 3ch)	Medium 미선택 (2 단계로 수렴)	-



[그림 10] Pareto front 비교: Early-Exit vs AdaPath (WM-811K)

결론적으로 Medium 경로는 ① 후단부 연산이 크고 ② 입력 난이도 분포가 넓어 Early 와 Full 사이의 중간 확신 구간이 많을 때 효과가 있다. MNIST 처럼 대부분이 쉬운 데이터에서는 이득이 작고, 모델 A 에서는 최적화가 Medium 을 자동으로 제외한다. 따라서 임계값을 Python 단계에서 데이터에 맞춰 결정하는 본 설계 흐름이 '2 단계/3 단계 중 어느 쪽이 유리한가'까지 자동으로 판단하는 구조라는 점이 의미가 있다.

3.9 자유주제: WM-811K 웨이퍼맵 결함 분류 적용

반도체 제조 공정의 웨이퍼맵 결함 패턴(WM-811K, 811,457 장 중 라벨 172,950 장)에서 정상(none), Center, Edge-Ring, Scratch 4 개 클래스를 클래스당 같은 개수(가장 적은 Scratch 기준 약 1,190 장)로 균형 추출하고, 웨이퍼 맵(웨이퍼 밖/정상 die/불량 die)을 28×28 로 리사이즈한 뒤 8:2 로 학습·테스트를 나누었다. 모델 B 구조(출력 4-class)에 같은 AdaPath 기법을 적용하였으며, 데이터 준비·학습·INT8·RTL 생성은 MNIST 와 동일한 코드로 수행하였다.

[표 13] WM-811K 적용 전·후 비교 (Baseline_WM = 항상 Hard)

항목	Baseline_WM	AdaPath_WM	변화
정확도 (테스트 954 장)	81.55 %	81.24 %	-0.31 %p
모드 비율 (E/M/H)	0 / 0 / 100 %	42.0 / 15.9 / 42.0 %	-
평균 MAC / 이미지	82,304	62,848	-23.6 %
Cell Area (μm²)	75,765	92,892	+22.6 %
Fmax (MHz)	540	551	-
평균 cycle / 이미지	985	944	-4.1 %
추론당 에너지 (nJ, 추정)	33.4	26.2	-21.7 %

WM 에너지: 동일 구조(모델 B)의 모드별 게이트레벨 실측 에너지에 WM-811K 모드 비율을 가중한 값 (출력 클래스 수만 10→4 로 달라 FC 일부만 차이).

4. 결과 및 기대효과

4.1 결과 요약

[표 14] 최종 결과 요약 (지정주제: Reference 대비 AdaPath_A)

지표	Reference	AdaPath_A	변화
정확도 (동일 1,000 장)	97.0 %	97.0 %	유지
평균 cycle / 추론	1,334	852	-36.1 %
Fmax (MHz)	623	598	-4.1 %
추론 지연 @Fmax (ns)	2,140	1,425	-33.4 %
평균 MAC / 추론	58,080	44,275	-23.8 %
추론당 에너지 (nJ)	17.7	16.2	-8.9 %
Cell Area (μm²)	32,085	49,858	+55.4 %

4.2 한계 및 향후 개선

· **순차 소자 전력:** 남은 에너지의 대부분이 레지스터·클럭 전력이다. 모드 신호로 후단부 레지스터의 클럭을 끄는 클럭 게이팅(ICG 셀)을 적용하면 Easy 모드 에너지를 추가로 줄일 수 있다.

· **면적:** Feature Map Buffer 와 Early Classifier 가 면적 증가의 주요인이다. 버퍼를 SRAM 매크로로 바꾸거나 Early Classifier 를 Full FC 와 시분할 공유하면 오버헤드를 줄일 수 있다.

· **Pre-layout 결과:** 본 결과는 합성 후 STA·전력이며, 배치·배선 후(post-layout) 검증이 필요하다.

· **임계값 가변화:** difficulty_decoder 는 레지스터 임계값 모드(USE_REG_TH)를 지원하므로, 런타임에 정확도-에너지 동작점을 바꾸는 DVFS 형 운용으로 확장할 수 있다.

4.3 기대효과

· **엣지 AI 저전력화:** 판단 회로(전체 면적의 1 % 미만) 추가만으로 쉬운 입력의 연산을 생략하여, 배터리 기반 엣지 기기의 추론당 에너지와 지연을 함께 줄인다.

· **반도체 공정 검사 적용:** 웨이퍼맵 검사처럼 정상·명확한 패턴이 다수인 공정 데이터에서 실시간 인라인 분류의 처리량을 높이고, 애매한 패턴만 정밀 경로로 보내는 계층적 검사에 활용할 수 있다.

· **설계 방법론:** Python 임계값 탐색 → INT8 Golden → 파라미터화 RTL → ORFS 합성 → 게이트레벨 VCD 전력까지 하나의 흐름으로 연결하여, 다른 모델·데이터셋에도 같은 절차로 적용 가능함을 보였다.

※ 참고문헌

[1] S. Teerapittayanon, B. McDanel, H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," ICPR, 2016.

[2] M.-J. Wu, J.-S. R. Jang, J.-L. Chen, "Wafer map failure pattern recognition and similarity ranking for large-scale data sets," IEEE Trans. Semiconductor Manufacturing, 2015. (WM-811K)

[3] Y. LeCun et al., "Gradient-based learning applied to document recognition," Proc. IEEE, 1998. (MNIST)

[4] L. T. Clark et al., "ASAP7: A 7-nm finFET predictive process design kit," Microelectronics Journal, 2016.

[5] T. Ajayi et al., "Toward an open-source digital flow: First learnings from the OpenROAD project," DAC, 2019.